Reflected XSS in a JavaScript URL with Some Characters Blocked

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting
Category	URL-Based Injection
Injection Context	JavaScript URL
Security Restrictions	Character Filtering
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy

2. Description

This vulnerability occurs when an application places user-controlled input into a URL that is interpreted by the browser as a JavaScript URL.

JavaScript URLs use the `javascript:` scheme, which can execute JavaScript when processed in an executable navigation context.

Although the application blocks certain characters, incomplete filtering may leave alternative ways to construct an unsafe URL.

The lab demonstrates why character blacklists are insufficient for protecting URL-based injection contexts.

3. Root Cause

The root cause is inadequate validation of user-controlled URL values.

Contributing factors include:

- Allowing untrusted input to influence executable URL schemes.

- Relying on character-based filtering.
- Missing URL scheme validation.
- Failure to distinguish safe navigation URLs from executable URLs.

4. Proof of Concept (PoC)

Authorized lab testing methodology:

1. Open the assigned lab.
2. Identify the reflected URL parameter.
3. Locate the HTML element containing the URL.
4. Inspect the generated URL in the response.
5. Determine which characters and URL components are filtered.
6. Examine whether the browser treats the resulting value as an executable JavaScript URL.
7. Test a harmless execution proof in the authorized environment.
8. Record the final URL structure and observed browser behavior.

Example URL context:

```
None
<a href="USER_CONTROLLED_URL">Open</a>
```

The example shows the injection context, not an executable payload.

5. Security Impact

If successfully exploited, the vulnerability may allow:

- JavaScript execution in the application's origin.
- Manipulation of page content.
- Unauthorized actions using the victim's authenticated session.
- Exposure of client-side information.
- Malicious navigation or deceptive content.

6. Recommended Remediation

- Reject dangerous URL schemes such as `javascript:`.
- Allow only explicitly approved URL schemes, typically HTTPS and HTTP where required.
- Parse and validate URLs using reliable URL parsing APIs.
- Apply HTML attribute encoding when rendering URLs.
- Avoid relying on character blacklists.

- Use safe DOM APIs.
- Apply a restrictive CSP as an additional security measure.
- Add tests for URL normalization and encoding variations.

7. Verification

1. Retest the original input.
2. Verify that dangerous URL schemes are rejected.
3. Test URL encoding and normalization variations.
4. Confirm that legitimate links continue working.
5. Inspect the final DOM and browser navigation behavior.

8. References

- PortSwigger: XSS Contexts
- MDN: URL API
- OWASP XSS Prevention Cheat Sheet

9. Disclaimer

For educational purposes and authorized security testing only.